

Atividade: Algoritmos de Escalonamento de CPU

Para cada um dos algoritmos abaixo, reflita com as questões propostas. Além disso, busque identificar para qual tipo de sistema o algoritmo se torna mais adequado.

1. Escalonamento – *First Come, First Served (FCFS) (Primeiro a chegar, primeiro a ser servido)*

O algoritmo *First Come, First Served* (FCFS) é o mais simples e intuitivo de todos. Ele organiza os processos em uma fila de prontos na ordem em que chegam. Quando a CPU é liberada, o primeiro processo da fila é selecionado e executado até que finalize ou entre em estado de espera (por exemplo, aguardando uma operação de E/S). O FCFS é não preemptivo, ou seja, o processo mantém o uso da CPU até que termine.

Questões para reflexão:

- Que impacto o FCFS teria em um ambiente com processos de duração muito longa em comparação aos de duração muito curta?
- De que forma a ausência de preempção pode influenciar a justiça e o desempenho geral do sistema?

Escalonamento – *Shortest Job First (SJF) (Tarefa mais curta primeiro) / Shortest Remaining Time First (SRTF) (Tempo restante mais curto em seguida)*

O algoritmo *Shortest Job First* (SJF) prioriza a execução do processo com o menor tempo de CPU estimado. Ele é considerado ótimo em termos de tempo médio de espera, pois reduz o acúmulo de processos curtos esperando por longos. Entretanto, depende de uma estimativa precisa da duração de cada processo, o que nem sempre é possível.

A versão preemptiva do SJF é chamada *Shortest Remaining Time First* (SRTF). Nesse caso, se um novo processo chega com tempo estimado menor que o tempo restante do processo atual, ocorre preempção e o novo processo é escalonado imediatamente. Essa estratégia melhora a responsividade, mas pode aumentar a sobrecarga de trocas de contexto.

Questões para reflexão:

- Quais seriam as consequências se as estimativas de tempo forem imprecisas ou desatualizadas?
- Como o comportamento do SRTF pode afetar o equilíbrio entre processos longos e curtos?

3. Escalonamento – *Round Robin (RR) (Chaveamento circular)*

O algoritmo *Round Robin* atribui a cada processo um intervalo fixo de tempo, chamado *quantum*, para usar a CPU. Quando o quantum expira, o processo é interrompido (preemptado) e colocado no fim da fila de prontos, dando lugar ao próximo processo. Assim, todos os processos recebem oportunidades regulares de execução.

A escolha adequada do quantum é crucial. Um quantum muito pequeno gera muitas trocas de contexto, reduzindo a eficiência do sistema. Por outro lado, um quantum muito longo diminui a interatividade, fazendo o sistema se comportar de modo semelhante ao FCFS. O RR promove uma sensação de justiça e é a base para escalonadores modernos de sistemas multitarefa.

Questões para reflexão:

- Como o tamanho do quantum influencia a eficiência e a percepção de responsividade do sistema?
- De que forma o algoritmo equilibra o tempo de resposta dos usuários com o custo de trocas de contexto?

4. Escalonamento – *Por Prioridades*

O escalonamento por prioridades define que cada processo possui um nível de importância representado por um número de prioridade. O escalonador seleciona sempre o processo com maior prioridade para execução. Esse modelo pode ser implementado de forma não preemptiva (o processo atual só libera a CPU voluntariamente) ou preemptiva (processos de prioridade superior podem interromper outros em execução).

As prioridades podem ser fixas, quando atribuídas no início e mantidas até o fim da execução, ou dinâmicas, quando o sistema operacional as ajusta conforme o comportamento do processo — por exemplo, reduzindo a prioridade de processos que usam muita CPU ou aumentando a de processos que esperam por longos períodos. Esse modelo oferece flexibilidade e controle, mas pode levar à inanição (starvation) de tarefas menos prioritárias se o sistema não aplicar políticas de envelhecimento.

Questões para reflexão:

- Como o ajuste dinâmico de prioridades pode contribuir para a justiça entre processos?
- Quais estratégias podem ser aplicadas para evitar a inanição em um sistema de prioridades?

5. Escalonamento – *Por Loteria*

O escalonamento por loteria é um método probabilístico em que cada processo recebe uma quantidade de bilhetes (tickets) que representam suas chances de ser escolhido para usar a CPU. A cada decisão de escalonamento, um sorteio é realizado, e o processo dono do bilhete sorteado é executado. Processos com mais bilhetes têm maior probabilidade de uso da CPU, mas todos mantêm uma chance de execução, garantindo justiça estatística ao longo do tempo.

Questões para reflexão:

- Em que tipo de sistema a aleatoriedade controlada pode melhorar a justiça no uso da CPU?
- Que vantagens/desvantagens surgem entre previsibilidade e flexibilidade no uso do escalonamento por loteria?

6. Escalonamento – *Sistemas de Tempo Real*

Os sistemas de tempo real são projetados para garantir que certas tarefas sejam concluídas dentro de prazos rígidos de tempo, chamados *deadlines*. O objetivo principal não é maximizar o desempenho médio, mas assegurar que tarefas críticas cumpram seus prazos, mesmo sob alta carga. O descumprimento desses prazos pode causar falhas sérias, como em sistemas de controle industrial, dispositivos médicos ou aviação.

Os escalonadores de tempo real são geralmente determinísticos e baseados em prioridades, podendo empregar políticas fixas ou dinâmicas conforme a periodicidade das tarefas. Esses sistemas exigem monitoramento constante, previsibilidade e controle preciso sobre a utilização da CPU.

Questão para reflexão:

- Por que a previsibilidade é mais importante que o desempenho médio em um sistema de tempo real?